Caching

Load balancing helps you scale horizontally across an ever-increasing number of servers, but caching will enable you to make vastly better use of the resources you already have as well as making otherwise unattainable product requirements feasible. Caches take advantage of the locality of reference principle: recently requested data is likely to be requested again. They are used in almost every computing layer: hardware, operating systems, web browsers, web applications, and more. A cache is like short-term memory: it has a limited amount of space, but is typically faster than the original data source and contains the most recently accessed items. Caches can exist at all levels in architecture, but are often found at the level nearest to the front end, where they are implemented to return data quickly without taxing downstream levels.

Application server cache

Placing a cache directly on a request layer node enables the local storage of response data. Each time a request is made to the service, the node will quickly return locally cached data if it exists. If it is not in the cache, the requesting node will fetch the data from the disk. The cache on one request layer node could also be located both in memory (which is very fast) and on the node's local disk (faster than going to network storage).

What happens when you expand this to many nodes? If the request layer is expanded to multiple nodes, it's still quite possible to have each node host its own cache. However, if your load balancer randomly distributes requests across the nodes, the same request will go to different nodes, thus increasing cache misses. Two choices for overcoming this hurdle are global caches and distributed caches.

Content Delivery (or Distribution) Network (CDN)

CDNs are a kind of cache that comes into play for sites serving large amounts of static media. In a typical CDN setup, a request will first ask the CDN for a piece of static media; the CDN will serve that content if it has it locally available. If it isn't available, the CDN will query the back-end servers for the file, cache it locally, and serve it to the requesting user.

If the system we are building is not large enough to have its own CDN, we can ease a future transition by serving the static media off a separate subdomain (e.g., static.yourservice.com) using a lightweight HTTP server like Nginx, and cut-over the DNS from your servers to a CDN later.

Cache Invalidation

While caching is fantastic, it requires some maintenance to keep the cache coherent with the source of truth (e.g., database). If the data is modified in the database, it should be invalidated in the cache; if not, this can cause inconsistent application behavior.

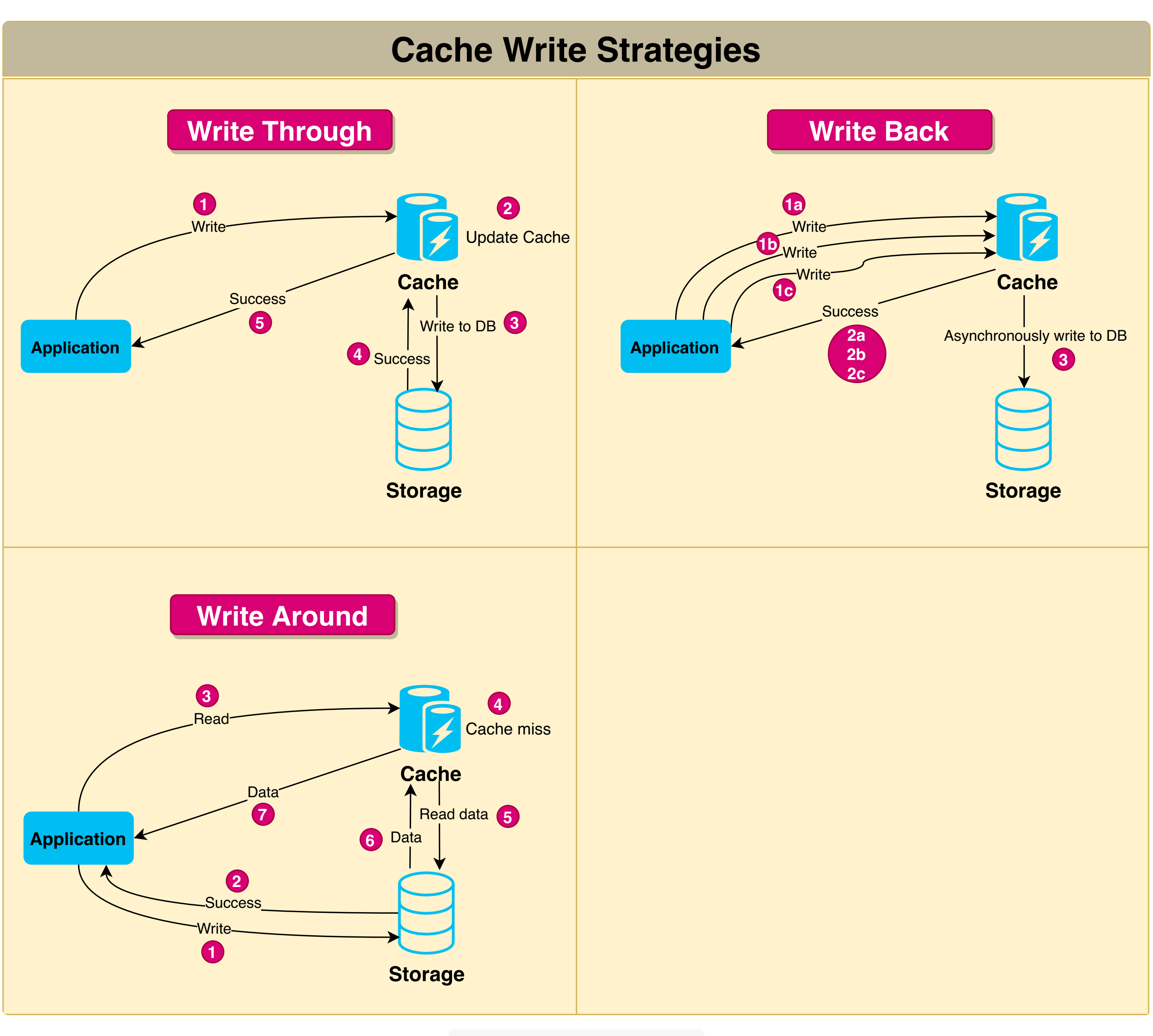
Solving this problem is known as cache invalidation; there are three main schemes that are used:

Write-through cache: Under this scheme, data is written into the cache and the corresponding database simultaneously. The cached data allows for fast retrieval and, since the same data gets written in the permanent storage, we will have complete data consistency between the cache and the storage. Also, this scheme ensures that nothing will get lost in case of a crash, power failure, or other system disruptions.

Although, write-through minimizes the risk of data loss, since every write operation must be done twice before returning success to the client, this scheme has the disadvantage of higher latency for write operations.

Write-around cache: This technique is similar to write-through cache, but data is written directly to permanent storage, bypassing the cache. This can reduce the cache being flooded with write operations that will not subsequently be re-read, but has the disadvantage that a read request for recently written data will create a “cache miss” and must be read from slower back-end storage and experience higher latency.

Write-back cache: Under this scheme, data is written to cache alone, and completion is immediately confirmed to the client. The write to the permanent storage is done after specified intervals or under certain conditions. This results in low-latency and high-throughput for write-intensive applications; however, this speed comes with the risk of data loss in case of a crash or other adverse event because the only copy of the written data is in the cache.

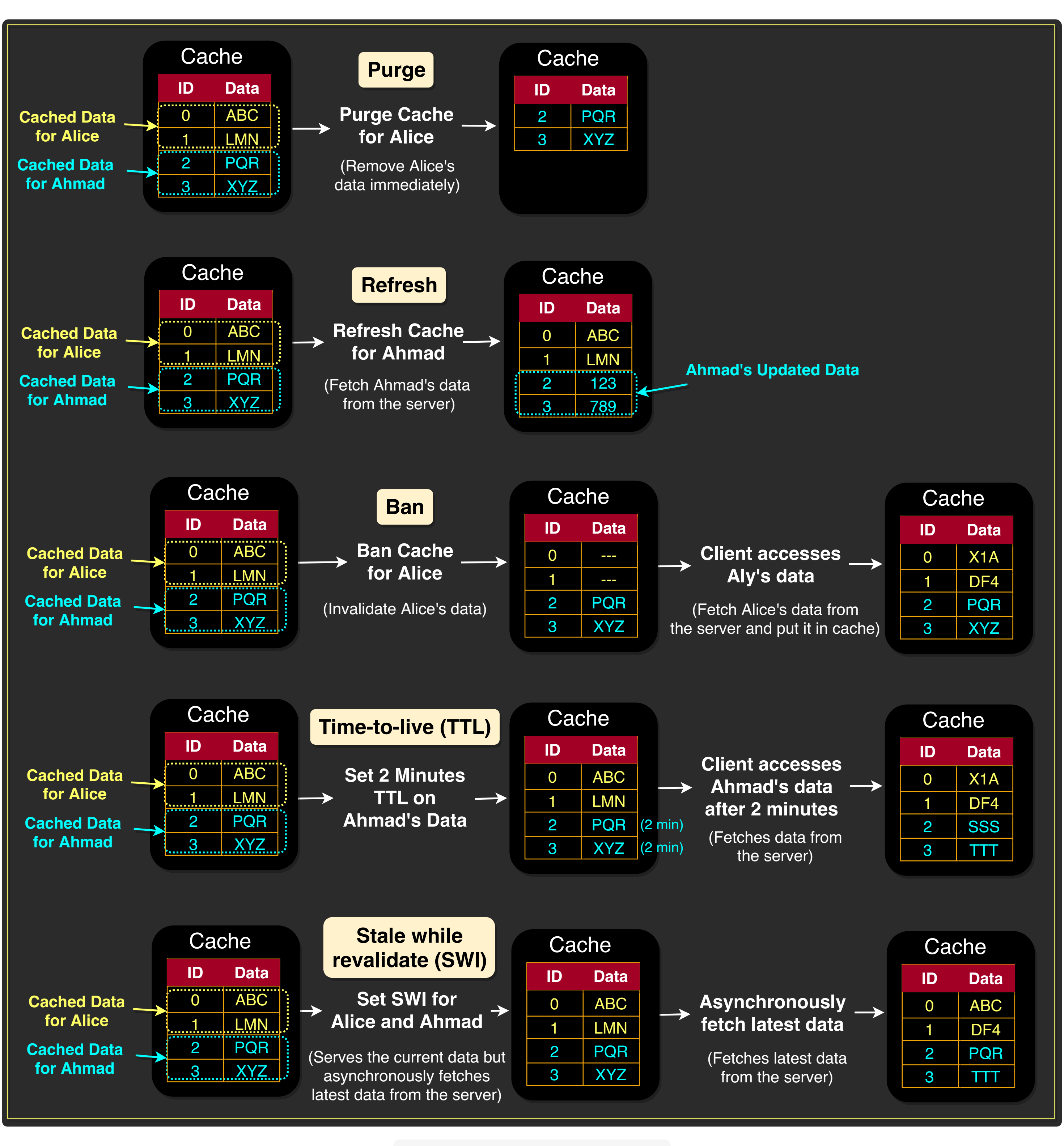

Cache Write Strategies
Write Through
1 Write
2 Update Cache
Cache
3 Write to DB
4 Success
5 Success
Application
Storage
Write Back
1a Write
1b Write
1c Write
Cache
Success
2a 2b 2c
Application
Asynchronously write to DB
3
Storage
Write Around
3 Read
4 Cache miss
Cache
Data 7
Read data 5
6 Data
Application
2 Success
Write 1
Storage

Cache Write Strategies

Cache Invalidation Methods

Here are the famous cache invalidation methods:

- **Purge:** The purge method removes cached content for a specific object, URL, or a set of URLs. It's typically used when there is an update or change to the content and the cached version is no longer valid. When a purge request is received, the cached content is immediately removed, and the next request for the content will be served directly from the origin server.
- **Refresh:** Fetches requested content from the origin server, even if cached content is available. When a refresh request is received, the cached content is updated with the latest version from the origin server, ensuring that the content is up-to-date. Unlike a purge, a refresh request doesn't remove the existing cached content; instead, it updates it with the latest version.
- **Ban:** The ban method invalidates cached content based on specific criteria, such as a URL pattern or header. When a ban request is received, any cached content that matches the specified criteria is immediately removed, and subsequent requests for the content will be served directly from the origin server.
- **Time-to-live** (TTL) **expiration:** This method involves setting a time-to-live value for cached content, after which the content is considered stale and must be refreshed. When a request is received for the content, the cache checks the time-to-live value and serves the cached content only if the value hasn't expired. If the value has expired, the cache fetches the latest version of the content from the origin server and caches it.
- **Stale-while-revalidate:** This method is used in web browsers and CDNs to serve stale content from the cache while the content is being updated in the background. When a request is received for a piece of content, the cached version is immediately served to the user, and an asynchronous request is made to the origin server to fetch the latest version of the content. Once the latest version is available, the cached version is updated. This method ensures that the user is always served content quickly, even if the cached version is slightly outdated.


Cache
ID Data
0 ABC
1 LMN
2 PQR
3 XYZ
Cached Data for Alice
Cached Data for Ahmad
Purge
Purge Cache for Alice
(Remove Alice's data immediately)
Cache
ID Data
2 PQR
3 XYZ
Refresh
Refresh Cache for Ahmad
(Fetch Ahmad's data from the server)
Cache
ID Data
0 ABC
1 LMN
2 123
3 789
Ahmad's Updated Data
Ban
Ban Cache for Alice
(Invalidate Alice's data)
Cache
ID Data
0 ---
1 ---
2 PQR
3 XYZ
Client accesses Aly's data
(Fetch Alice's data from the server and put it in cache)
Cache
ID Data
0 X1A
1 DF4
2 PQR
3 XYZ
Time-to-live (TTL)
Set 2 Minutes TTL on Ahmad's Data
Cache
ID Data
0 ABC
1 LMN
2 PQR (2 min)
3 XYZ (2 min)
Client accesses Ahmad's data after 2 minutes
(Fetches data from the server)
Cache
ID Data
0 X1A
1 DF4
2 SSS
3 TTT
Stale while revalidate (SWI)
Set SWI for Alice and Ahmad
(Serves the current data but asynchronously fetches latest data from the server)
Cache
ID Data
0 ABC
1 LMN
2 PQR
3 XYZ
Asynchronously fetch latest data
(Fetches latest data from the server)
Cache
ID Data
0 ABC
1 DF4
2 PQR
3 TTT

Cache Invalidation Methods

Cache read strategies

Here are the two famous cache read strategies:

Read through cache

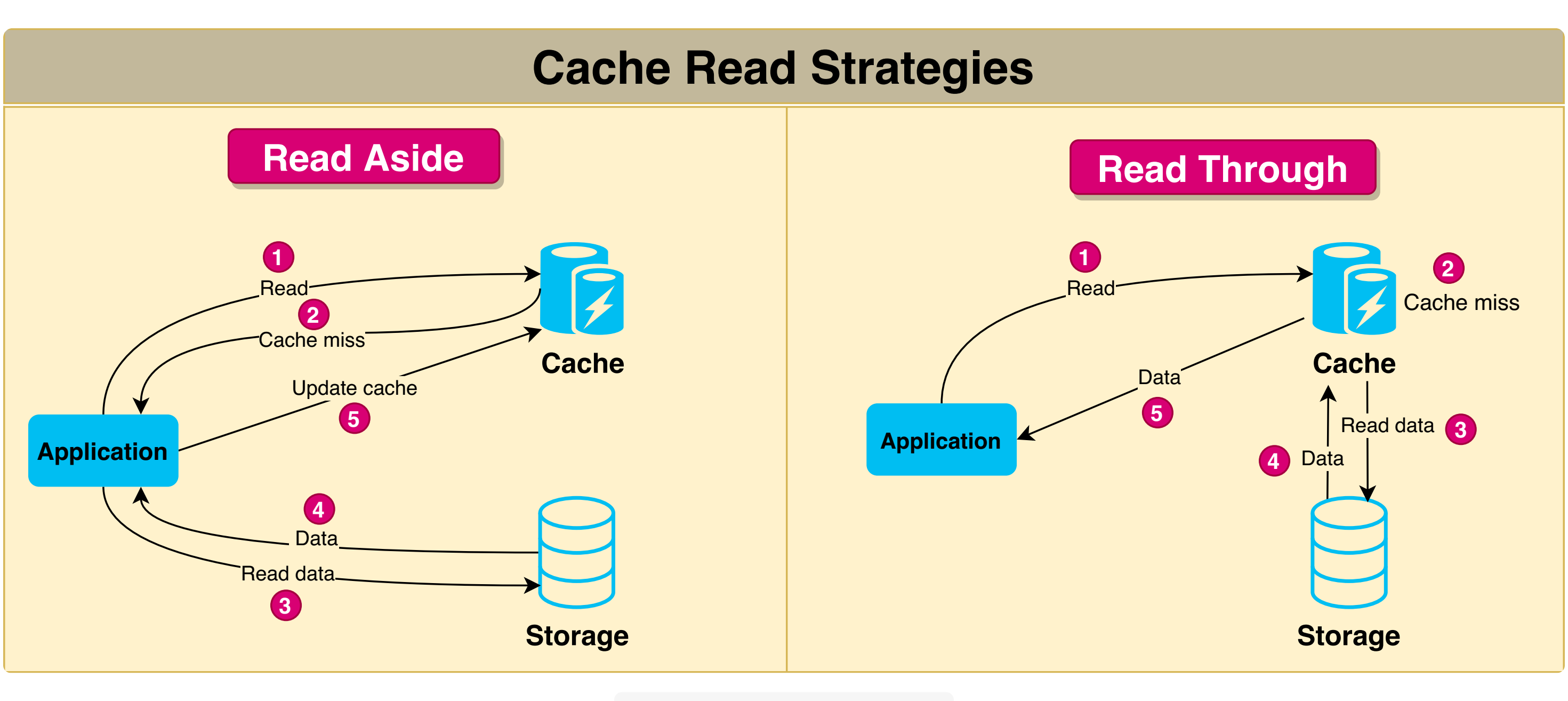
A read-through cache strategy is a caching mechanism where the cache itself is responsible for retrieving the data from the underlying data store when a cache miss occurs. In this strategy, the application requests data from the cache instead of the data store directly. If the requested data is not found in the cache (cache miss), the cache retrieves the data from the data store, updates the cache with the retrieved data, and returns the data to the application.

This approach helps to maintain consistency between the cache and the data store, as the cache is always responsible for retrieving and updating the data. It also simplifies the application code since the application doesn't need to handle cache misses and data retrieval logic. The read-through cache strategy can significantly improve performance in scenarios where data retrieval from the data store is expensive, and cache misses are relatively infrequent.

Read aside cache

A read-aside cache strategy, also known as **cache-aside** or **lazy-loading**, is a caching mechanism where the application is responsible for retrieving the data from the underlying data store when a cache miss occurs. In this strategy, the application first checks the cache for the requested data. If the data is found in the cache (cache hit), the application uses the cached data. However, if the data is not present in the cache (cache miss), the application retrieves the data from the data store, updates the cache with the retrieved data, and then uses the data.

The read-aside cache strategy provides better control over the caching process, as the application can decide when and how to update the cache. However, it also adds complexity to the application code, as the application must handle cache misses and data retrieval logic. This approach can be beneficial in scenarios where cache misses are relatively infrequent, and the application wants to optimize cache usage based on specific data access patterns.


Cache Read Strategies
Read Aside
1 Read
2 Cache miss
Update cache 5
Cache
Application
4 Data
Read data 3
Storage
Read Through
1 Read
2 Cache miss
Cache
Data 5
Read data 3
4 Data
Application
Storage

Cache Read Strategies

Cache eviction policies

Following are some of the most common cache eviction policies:

1. First In First Out (FIFO): The cache evicts the first block accessed first without any regard to how often or how many times it was accessed before.
2. Last In First Out (LIFO): The cache evicts the block accessed most recently first without any regard to how often or how many times it was accessed before.
3. Least Recently Used (LRU): Discards the least recently used items first.
4. Most Recently Used (MRU): Discards, in contrast to LRU, the most recently used items first.
5. Least Frequently Used (LFU): Counts how often an item is needed. Those that are used least often are discarded first.
6. Random Replacement (RR): Randomly selects a candidate item and discards it to make space when necessary.

Following links have some good discussion about caching:
[1] Cache
[2] Introduction to architecting systems

← Previous
Load Balancing

Next →
Data Partitioning

☐ Mark as Completed